

C++ Data Structures & STL

The Comprehensive Guide to Efficient
Programming

Deep Dive into the Standard Template Library

A Beginner to Intermediate Guide

Table of Contents

1. Introduction to Data Structures
2. Core Philosophy of C++ STL
3. The Memory Model & Big O Notation
4. Dynamic Arrays: The Vector Container
5. Sequential Access: Lists and Deques
6. LIFO and FIFO: Stacks and Queues
7. Sorted Storage: Sets and Multisets
8. Key-Value Mapping: Maps and Multimaps
9. Unordered Containers (Hash Tables)
10. Navigating Data: Iterators in Depth
11. STL Algorithms: Searching and Sorting
12. Functional STL: Predicates and Lambdas
13. Practical Use-Cases & Problem Solving
14. Performance Optimization and Best Practices
15. Conclusion and Advanced Resources

1. Introduction to Data Structures

At its core, computer science is about two things: processing data and storing data. A **Data Structure** is a specialized format for organizing, processing, retrieving, and storing data. While any data structure can technically hold data, the choice of structure determines the efficiency of the operations performed on that data.

In C++, fundamental types like `int` or `char` are basic building blocks. However, real-world applications require complex arrangements. For example:

- **Arrays:** Store elements in contiguous memory. Fast access, but fixed size.
- **Linked Lists:** Store elements in non-contiguous nodes. Fast insertion, but slow access.
- **Trees:** Hierarchical structures for rapid searching.

Analogy: Imagine a library. If books are scattered on the floor, finding one takes forever (Inefficient Data Structure). If books are alphabetized on shelves, finding one is instant (Efficient Data Structure).

2. Core Philosophy of C++ STL

The Standard Template Library (STL) is a set of C++ template classes to provide common programming data structures and functions such as lists, stacks, arrays, etc. It is a library of container classes, algorithms, and iterators. It is a generalized library and so, its components are parameterized.

The Three Pillars of STL

1. **Containers:** Objects that store data. Examples include `vector`, `list`, and `map`.
2. **Algorithms:** Procedures that act on data. Examples include `sort()`, `find()`, and `transform()`.
3. **Iterators:** The bridge between containers and algorithms. They allow algorithms to move through a container without knowing its internal structure.

Generic Programming: The STL is built on the concept of generic programming. This means you can use a `vector` to store `int`, `string`, or even your custom `User` class without rewriting the `vector` code.

3. The Memory Model & Big O Notation

Before diving into specific structures, we must understand how we measure efficiency. This is usually done using **Big O Notation**, which describes the upper bound of execution time or space used by an algorithm as a function of the input size (N).

Notation	Name	Description
$O(1)$	Constant	Time does not change with data size.
$O(\log N)$	Logarithmic	Time increases slowly (Binary Search).
$O(N)$	Linear	Time increases proportionally to data.
$O(N \log N)$	Linearithmic	Standard for efficient sorting.

C++ STL guarantees specific time complexities for its operations. When you use a `std::vector::push_back`, it is guaranteed to be **Amortized $O(1)$** , making C++ one of the fastest languages for data manipulation.

4. Dynamic Arrays: The Vector

The `std::vector` is the most commonly used container in C++. It is essentially a dynamic array that grows automatically when elements are added.

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v;

    // Adding elements
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);

    // Accessing elements
    std::cout << "First element: " << v[0] << std::endl;

    // Size vs Capacity
    std::cout << "Size: " << v.size() << std::endl;
    std::cout << "Capacity: " << v.capacity() << std::endl;

    return 0;
}
```

Why use Vector?

- **Contiguous Memory:** Elements are next to each other, making them cache-friendly.
- **Random Access:** You can access any element in $O(1)$ time using an index.

- **Safety:** Provides the `.at()` method which checks for out-of-bounds errors.

5. Sequential Access: Lists and Deques

`std::list`

Unlike vectors, `std::list` is implemented as a **Doubly Linked List**. Each element is a node containing the data and pointers to the previous and next nodes. Memory is not contiguous.

- **Fast Insert/Delete:** $O(1)$ if you have an iterator to the position.
- **No Random Access:** To find the 5th element, you must start from the beginning and count.

`std::deque`

Short for "Double-Ended Queue," a deque is like a hybrid between a vector and a list. It allows $O(1)$ insertion at both the front and the back, and it still provides random access.

```
#include <deque>
std::deque<int> d = {1, 2, 3};
d.push_front(0); // O(1)
d.push_back(4);  // O(1)
```


6. LIFO and FIFO: Stacks and Queues

These are "Container Adapters." They provide a restricted interface to other containers (like vector or deque) to enforce specific behavior.

std::stack (LIFO)

Last-In, First-Out. Imagine a stack of plates. You put the newest one on top and take the newest one off first.

- `push()`: Add to top
- `pop()`: Remove from top
- `top()`: Look at top item

std::queue (FIFO)

First-In, First-Out. Imagine a line at a grocery store. The first person to join the line is the first one served.

```
#include <queue>
std::queue<string> line;
line.push("Customer A");
line.push("Customer B");
std::cout << line.front(); // Customer A
line.pop();
```

7. Sorted Storage: Sets and Multisets

`std::set` is an associative container that contains a sorted set of unique objects. It is usually implemented using a **Red-Black Tree** (a type of self-balancing binary search tree).

Key Characteristics:

- **Uniqueness:** Duplicate values are discarded.
- **Automatic Sorting:** Elements are always stored in ascending order.
- **Search Speed:** $O(\log N)$ for searching, inserting, and deleting.

```
std::set<int> s;  
s.insert(5);  
s.insert(1);  
s.insert(5); // Ignored - duplicate  
// Set now contains: {1, 5}
```

`std::multiset`: Same as `set`, but allows duplicate values.

8. Key-Value Mapping: Maps and Multimaps

A `std::map` is an associative container that stores elements formed by a combination of a **Key** and a **Value**. The key is used to identify the element, and the value stores the data associated with that key.

```
#include <map>
#include <string>

std::map<int, std::string> directory;
directory[101] = "Alice";
directory[102] = "Bob";

// Searching for a key
if (directory.count(101)) {
    std::cout << "Found: " << directory[101];
}
```

Maps are sorted by Key. If you iterate through a map of names, the names will appear in alphabetical order regardless of the order you inserted them.

9. Unordered Containers (Hash Tables)

Since C++11, we have `std::unordered_map` and `std::unordered_set`. These use **Hash Tables** instead of trees. Unlike regular maps/sets, they do not keep elements in order.

Comparison:

Feature	<code>std::map</code>	<code>std::unordered_map</code>
Internal Structure	Red-Black Tree	Hash Table
Search Time	$O(\log N)$	$O(1)$ average
Iteration Order	Sorted by Key	Random/Undefined

Use unordered containers when you need maximum performance and don't care about the order of keys.

10. Navigating Data: Iterators in Depth

Iterators are objects that point to elements within a container. They act like smart pointers that know how to navigate the specific structure of the container.

Common Iterator Functions:

- `begin()`: Points to the first element.
- `end()`: Points to the position *after* the last element.
- `++it`: Moves to the next element.
- `*it`: Dereferences (gets the value) at the current position.

```
std::vector<int> v = {10, 20, 30};  
for (auto it = v.begin(); it != v.end(); ++it) {  
    std::cout << *it << " ";  
}
```

Iterators enable algorithms to remain separate from the implementation of the data structure.

11. STL Algorithms: Searching and Sorting

The `<algorithm>` header provides over 100 functions. These are highly optimized and should be used instead of manual loops whenever possible.

1. Sorting

`std::sort(v.begin(), v.end())` sorts elements in $O(N \log N)$ time.

2. Searching

- `std::find(v.begin(), v.end(), value)`: Linear search.
- `std::binary_search(sorted_v.begin(), sorted_v.end(), value)`: Returns true/false.

3. Modifying

- `std::reverse(v.begin(), v.end())`
- `std::fill(v.begin(), v.end(), 0)`

```
vector<int> v = {3, 1, 4, 1, 5};  
std::sort(v.begin(), v.end());  
// v is now {1, 1, 3, 4, 5}
```

12. Functional STL: Predicates and Lambdas

Modern C++ allows you to pass logic into algorithms using Lambda functions. This makes algorithms incredibly flexible.

```
std::vector<int> nums = {1, 2, 3, 4, 5, 6};

// Count how many numbers are even
int evens = std::count_if(nums.begin(), nums.end(), [](int n) {
    return n % 2 == 0;
});
```

The `[]` syntax defines a lambda. This allows you to write custom search or sort criteria on the fly, such as sorting a list of students by their Grade Point Average (GPA).

13. Practical Use-Cases & Problem Solving

Choosing the right data structure is half the battle in software engineering.

Scenario 1: Browser History

A "Forward" and "Back" button system is best implemented with two **Stacks** or a **Deque**.

Scenario 2: Video Game Leaderboard

To keep a list of scores sorted at all times with quick insertion of new scores, a **std::set** or **std::multiset** is ideal.

Scenario 3: Frequency Counter

If you need to count how many times each word appears in a book, a **std::map<string, int>** is the standard solution.

Rule of Thumb: Use `std::vector` by default. Only move to other structures if you have a specific bottleneck or requirement that vector doesn't handle well.

14. Performance Optimization and Best Practices

1. **Reserve Space:** If you know you will add 1000 items to a vector, use `v.reserve(1000)` to avoid multiple re-allocations.
2. **Pass by Reference:** When passing containers to functions, use `const std::vector<int>& v` to avoid copying the entire data structure.
3. **Use `emplace_back`:** `v.emplace_back(args)` can be more efficient than `v.push_back(item)` because it constructs the object in place.
4. **Prefer range-based for loops:** `for (auto& item : container)` is cleaner and less error-prone than manual index management.

15. Conclusion and Advanced Resources

The C++ STL is a massive toolbox. We have covered the essentials—the maps, vectors, and algorithms that power modern software. Mastering these tools will not only make you a more efficient coder but also prepare you for high-level technical interviews and complex systems development.

Next Steps:

- **Explore C++20 Ranges:** A newer, more powerful way to handle STL algorithms.
- **Memory Management:** Learn about Smart Pointers (`unique_ptr`, `shared_ptr`) to use with these containers.
- **Custom Templating:** Try writing your own Template class to understand how STL works under the hood.

Keep Coding. Keep Optimizing.